

07 Extreme Gradient Boosting

EP

2026-07-19

eXtreme Gradient Boosting (XGBoost)

```
library(targets)
library(tidymodels)
library(xgboost)
library(sf)
library(dplyr)

set.seed(399)
```

As seen in the Random Forest models, `n_days` is dominant predictor. For species richness, the full model has an ROC AUC of 0.956 dropping to 0.706 in the ND model (no `n_days`). The full Simpson model has an ROC AUC of 0.712, dropping to 0.662 in the ND model. City level breakdown shows consistency between cities in the full richness model (ROC AUC ranging from 0.981-0.920), whereas the ND richness model has more variability (0.795-0.648). This suggests that without the dominant predictor, `n_days`, the model does not generalise to unseen cities as well. Both Simpson models also see greater variation in their ROC AUC scores: full (0.810-0.674) and ND (0.728-0.605). Comparison of the success rate vs prediction rate AUC in final full richness model shows a small decrease (success 0.985 vs prediction 0.956) suggesting the model fits the training data well and can predict unseen data well. However, when removing `n_days` there is a bigger gap (success 0.835 vs prediction 0.706) suggesting more overfitting now the model has to rely on weaker, potentially noisier, signals. Both the Simpson models have large gaps: full (success 0.792 vs prediction 0.712) and ND (success 0.722 vs prediction 0.662).

However, removing `n_days` highlights `weighted_temp`, `pct_greenpace`, and `weighted_precip` are drivers of richness diversity. `pct_greenpace` and `avg_area_ha` are top drivers of Simpson diversity. This is in agreement with the previous Random Forest results.

Load Data

```
# Uncomment if using pipeline data:

#setwd("~/CSCT_Pipeline")

#grid_model_all <- tar_read(grid_model_all)

#df <- grid_model_all %>%
  #sf::st_drop_geometry()
```

```

#If running with downloaded grid data use:

grid_model_all <- st_read("Data/grid_model.gpkg", quiet = TRUE)

df <- grid_model_all %>%
  sf::st_drop_geometry()

# Adjust Simpson Scores
n <- nrow(df)
df$simpson_adj <- (df$simpson * (n - 1) + 0.5) / n

# Set predictors
predictors <- c("n_days", "dtm_elev", "avg_imperv", "rofsw_risk", "rofrs_risk",
  "avg_area_ha", "treeht", "weighted_precip", "weighted_temp",
  "dist_to_river", "pct_greenpace")

```

Classification and CV Fold Setup

```

#Classification

df <- df %>%
  mutate(
    richness_class = factor(if_else(sp_richness >= median(sp_richness, na.rm = TRUE),
      "high", "low"),
      levels = c("low", "high")),
    simpson_class = factor(if_else(simpson_adj >= median(simpson_adj, na.rm = TRUE),
      "high", "low"),
      levels = c("low", "high")))

#Cross-validation folds
folds <- group_vfold_cv(df, group = city)

fold_city_lookup <- folds %>%
  mutate(city = purrr::map_chr(splits, ~ as.character(unique(assessment(.x)$city)))) %>%
  select(id, city)

```

Model Specification and Recipe

```

#XGBoost Specification

xgb_spec_class <- boost_tree(
  trees          = tune(),
  tree_depth     = tune(),
  learn_rate     = tune(),
  loss_reduction = tune(),
  mtry           = tune(),
  min_n          = tune(),
  sample_size    = tune()) %>%

```

```

set_engine("xgboost") %>%
set_mode("classification")

#Recipes and workflows
richness_recipe_xgb <- recipe(
  as.formula(paste("richness_class ~", paste(predictors, collapse = " + "))),
  data = df) %>%
  # replace missing values with "unknown"
  step_unknown(all_nominal_predictors()) %>%
  # remove observations missing numerical predictors
  step_naomit(all_numeric_predictors(), all_outcomes()) %>%
  # allow previously unseen categorical levels in new data
  step_novel(all_nominal_predictors()) %>%
  # convert categorical predictors into dummy variables
  step_dummy(all_nominal_predictors())

richness_wf_xgb <- workflow() %>%
  add_recipe(richness_recipe_xgb) %>%
  add_model(xgb_spec_class)

simpson_recipe_xgb <- recipe(
  as.formula(paste("simpson_class ~", paste(predictors, collapse = " + "))),
  data = df) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_naomit(all_numeric_predictors(), all_outcomes()) %>%
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors())

simpson_wf_xgb <- workflow() %>%
  add_recipe(simpson_recipe_xgb) %>%
  add_model(xgb_spec_class)

```

Tuning Setup

```

xgb_grid <- grid_space_filling(
  trees(range = c(100, 1000)), # Number of boosting trees
  tree_depth(range = c(2, 8)), # Max depth of each tree
  learn_rate(range = c(-3, -0.5)), # Learning rate, log10 scale
  loss_reduction(range = c(-5, 1.5)), # Min improvement for further tree split (also log10 scale)
  finalize(mtry(range = c(2, 8)), df[, predictors]), # Number of predictors at each split, finalised ba
  min_n(range = c(2, 15)), # Min obs at each node
  sample_prop(range = c(0.5, 1.0)), # Proportion of training observations
  size = 30,
  type = "latin_hypercube") # 30 combos using latin hypercube sampling

#Latin hypercube sampling used for efficient, equal parameter testing without testing every possibility

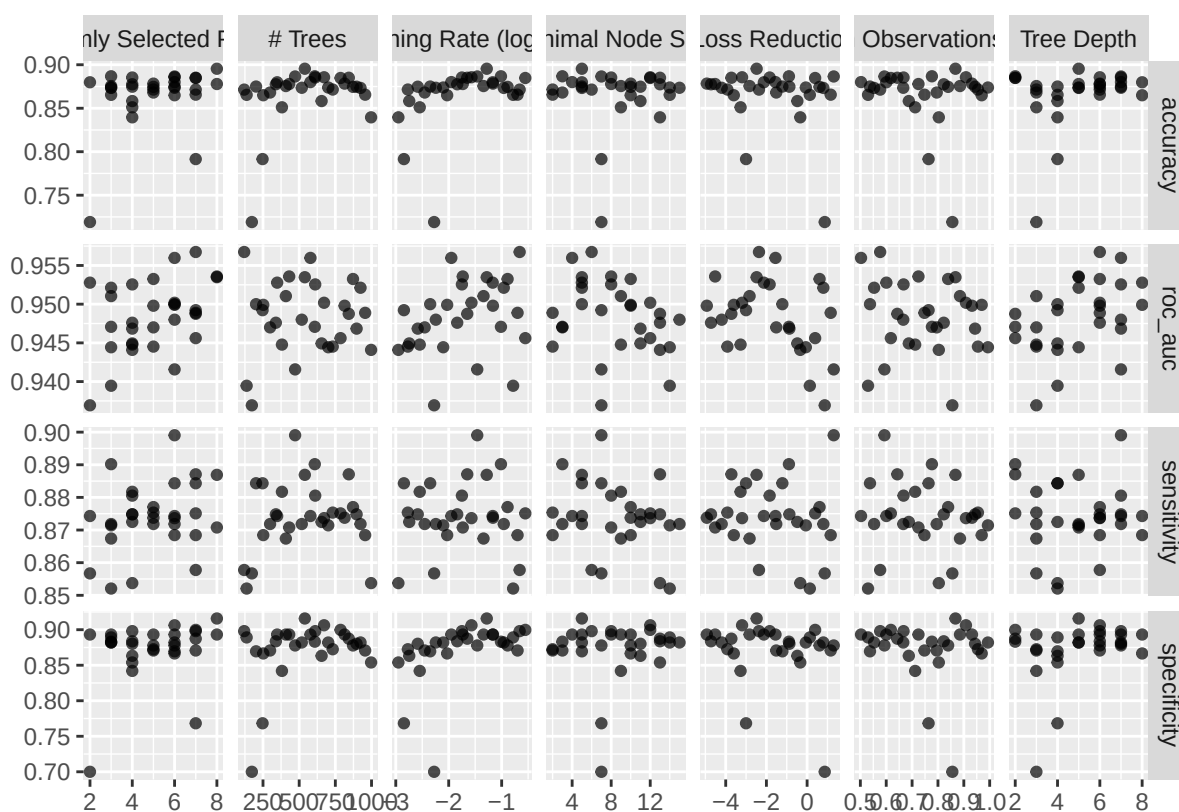
```

Hyperparameter Tuning

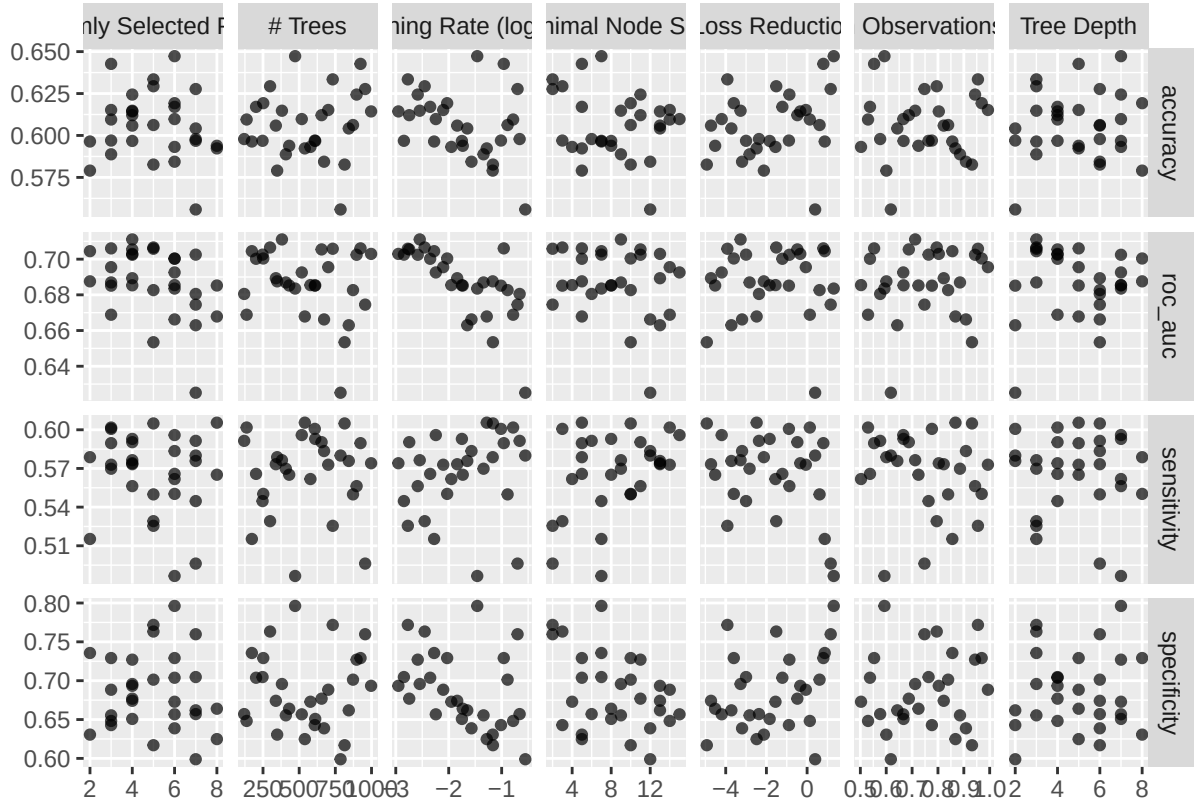
```
richness_res_xgb <- tune_grid(
  richness_wf_xgb,
  resamples = folds,
  grid       = xgb_grid,
  metrics    = metric_set(roc_auc, accuracy, sensitivity, specificity))

simpson_res_xgb <- tune_grid(
  simpson_wf_xgb,
  resamples = folds,
  grid       = xgb_grid,
  metrics    = metric_set(roc_auc, accuracy, sensitivity, specificity))

autoplot(richness_res_xgb)
```



```
autoplot(simpson_res_xgb)
```



Select Best

```
show_best(richness_res_xgb, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, tree_depth, learn_rate, loss_reduction,
         sample_size, trees, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(caption = "Top 5 hyperparameter combinations:
                  species richness class (XGBoost, ranked by ROC AUC)")
```

Table 1: Top 5 hyperparameter combinations: species richness class (XGBoost, ranked by ROC AUC)

mtry	min_n	tree_depth	learn_rate	loss_reduction	sample_size	trees	Mean ROC	
							AUC	Std. error
7	6	6	0.2186334	0.0043208	0.5766876	120	0.957	0.009
6	4	7	0.0112078	0.0279783	0.5026014	578	0.956	0.010
8	8	5	0.0184371	0.0000302	0.7247171	431	0.954	0.010
8	5	5	0.0523601	0.0033148	0.8677105	539	0.953	0.010
5	10	6	0.1294866	4.0976573	0.8393988	872	0.953	0.012

```
show_best(simpson_res_xgb, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, tree_depth, learn_rate, loss_reduction,
         sample_size, trees, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(caption = "Top 5 hyperparameter combinations:
                  Simpson diversity class (XGBoost, ranked by ROC AUC)")
```

Table 2: Top 5 hyperparameter combinations: Simpson diversity class (XGBoost, ranked by ROC AUC)

mtry	min_n	tree_depth	learn_rate	loss_reduction	sample_size	trees	Mean ROC AUC	Std. error
4	9	3	0.0028402	0.0005382	0.7119274	381	0.711	0.027
5	3	3	0.0035202	0.0304874	0.7953771	299	0.707	0.022
3	5	5	0.1096675	6.3024466	0.5531246	924	0.706	0.026
5	2	3	0.0016925	0.0001210	0.9546309	732	0.706	0.023
4	11	4	0.0017983	0.3320400	0.6873552	655	0.705	0.026

```
best_richness_xgb <- select_best(richness_res_xgb, metric = "roc_auc")
best_simpson_xgb <- select_best(simpson_res_xgb, metric = "roc_auc")
```

Prediction Rate AUC

```
# Get overall mean ROC AUC for richness and Simpson
overall_richness_xgb <- collect_metrics(richness_res_xgb) %>%
  filter(.metric == "roc_auc") %>%
  semi_join(
    best_richness_xgb,
    by = names(best_richness_xgb)) %>%
  pull(mean)

overall_simpson_xgb <- collect_metrics(simpson_res_xgb) %>%
  filter(.metric == "roc_auc") %>%
  semi_join(
    best_simpson_xgb,
    by = names(best_simpson_xgb)) %>%
  pull(mean)

# Get city based ROC AUC scores (by fold, then attach city name)
city_richness_xgb <- collect_metrics(
  richness_res_xgb,
  summarize = FALSE) %>%
  filter(.metric == "roc_auc") %>%
  semi_join(
    best_richness_xgb,
    by = names(best_richness_xgb)) %>%
  left_join(fold_city_lookup, by = "id") %>%
  select(city, richness_auc = .estimate)
```

```

city_simpson_xgb <- collect_metrics(
  simpson_res_xgb,
  summarize = FALSE) %>%
filter(.metric == "roc_auc") %>%
semi_join(
  best_simpson_xgb,
  by = names(best_simpson_xgb)) %>%
left_join(fold_city_lookup, by = "id") %>%
select(city, simpson_auc = .estimate)

# Combine richness and Simpson city scores
city_auc_xgb <- full_join(city_richness_xgb, city_simpson_xgb,
  by = "city")

# Create row for overall score
overall_auc_xgb <- tibble(city = "Overall",
  richness_auc = overall_richness_xgb,
  simpson_auc = overall_simpson_xgb)

# Combine and create table
classification_auc_table_xgb <- bind_rows(
  overall_auc_xgb, city_auc_xgb) %>%
mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
rename(
  "City (Held-Out)" = city,
  "Richness ROC AUC" = richness_auc,
  "Simpson ROC AUC" = simpson_auc)

classification_auc_table_xgb %>%
knitr::kable(
  caption = paste(
    "Overall and city-level cross-validated ROC AUC for",
    "species richness and Simpson diversity (XGBoost)"))

```

Table 3: Overall and city-level cross-validated ROC AUC for species richness and Simpson diversity (XGBoost)

City (Held-Out)	Richness ROC AUC	Simpson ROC AUC
Overall	0.957	0.711
Nottingham	0.926	0.669
Exeter	0.960	0.703
Newcastle	0.962	0.658
Bristol	0.979	0.715
Gateshead	0.957	0.810

Final Model

```

final_richness_wf_xgb <- finalize_workflow(richness_wf_xgb, best_richness_xgb)
richness_fit_xgb <- fit(final_richness_wf_xgb, data = df)

```

```
final_simpson_wf_xgb <- finalize_workflow(simpson_wf_xgb, best_simpson_xgb)
simpson_fit_xgb <- fit(final_simpson_wf_xgb, data = df)
```

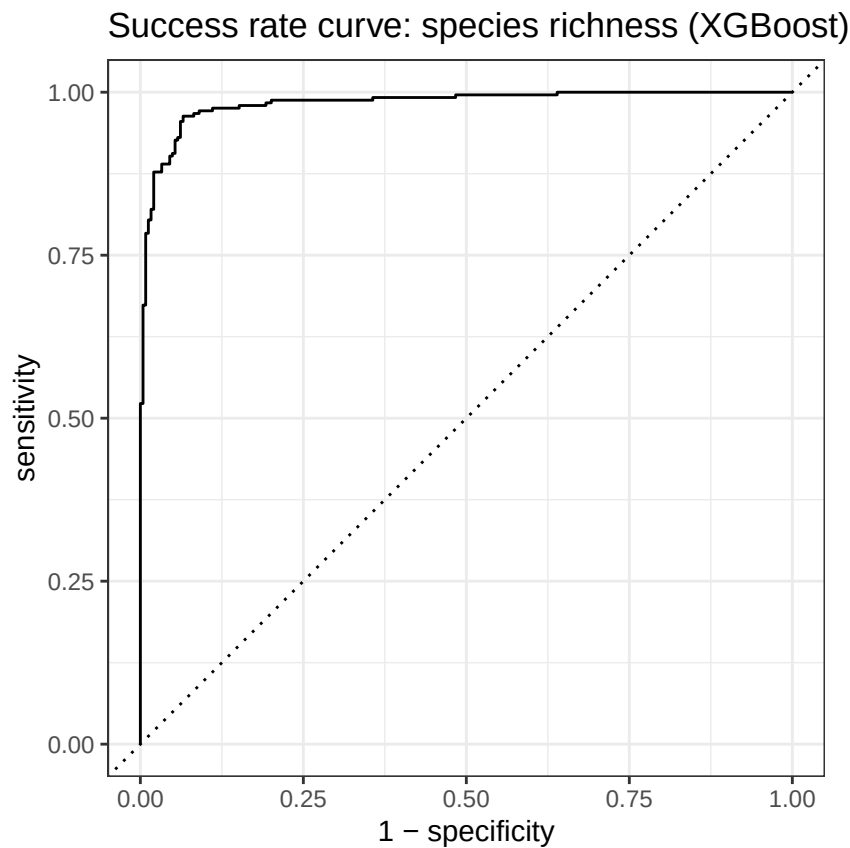
Success Rate AUC

```
richness_success_xgb <- augment(richness_fit_xgb, new_data = df)
simpson_success_xgb <- augment(simpson_fit_xgb, new_data = df)

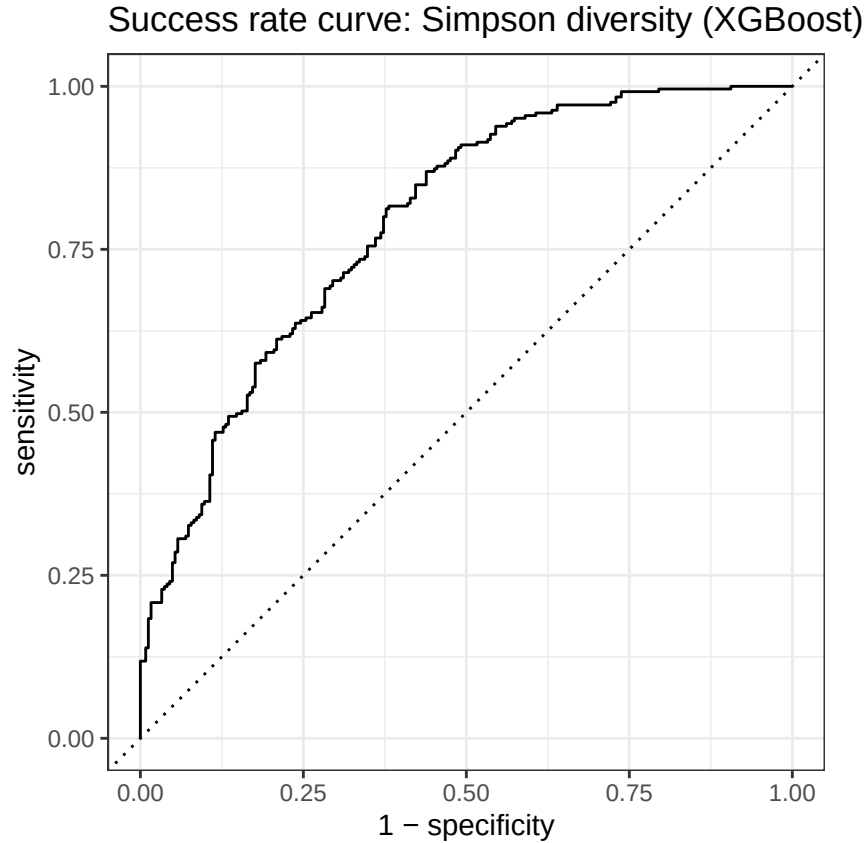
success_rate_richness_xgb <- roc_auc(richness_success_xgb, truth = richness_class,
                                     .pred_high, event_level = "second")
success_rate_simpson_xgb <- roc_auc(simpson_success_xgb, truth = simpson_class,
                                     .pred_high, event_level = "second")
```

ROC Curves

```
roc_curve(richness_success_xgb, truth = richness_class, .pred_high, event_level = "second") %>%
  autoplot() + labs(title = "Success rate curve: species richness (XGBoost)")
```



```
roc_curve(simpson_success_xgb, truth = simpson_class, .pred_high, event_level = "second") %>%
  autoplot() + labs(title = "Success rate curve: Simpson diversity (XGBoost)")
```

Summary Table

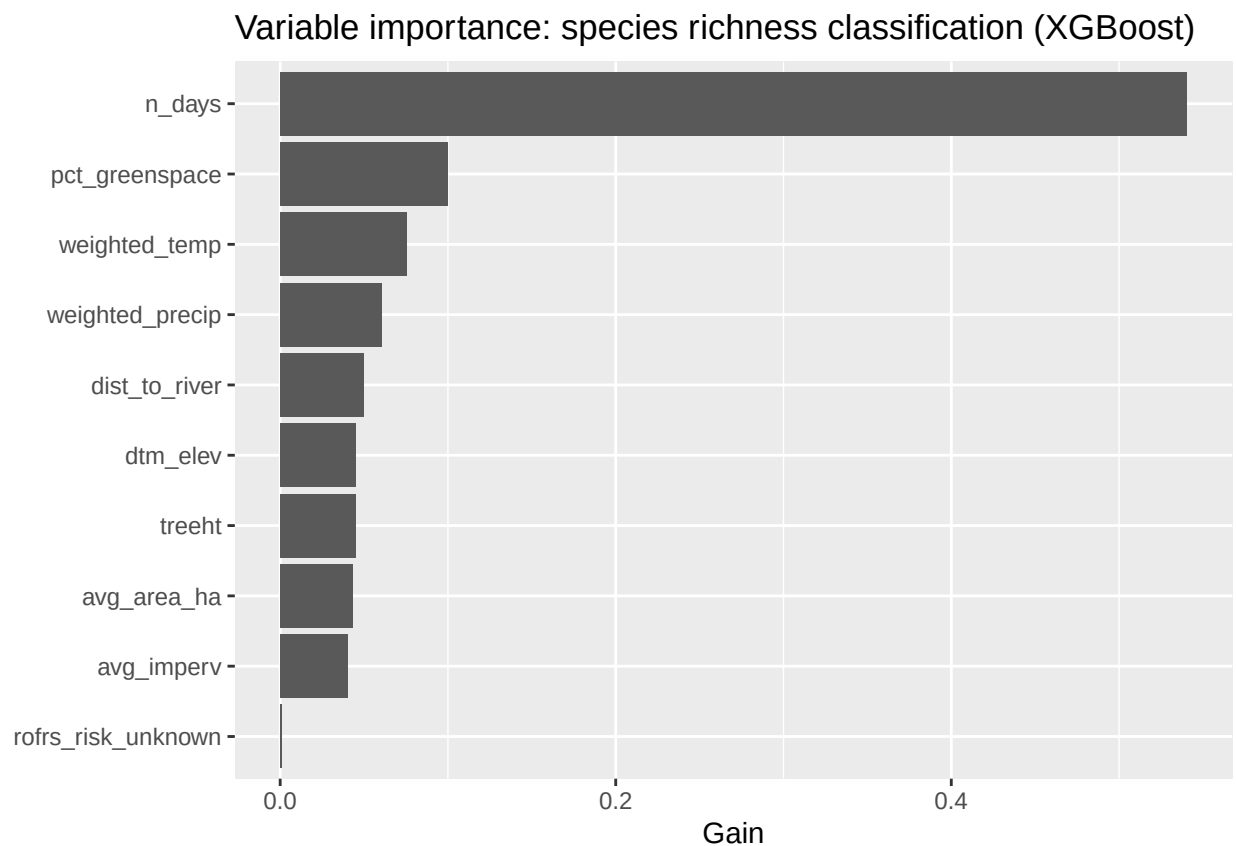
```
tibble::tibble(
  Outcome = c("Species richness", "Simpson diversity"),
  `Success rate AUC` = c(success_rate_richness_xgb$.estimate,
                        success_rate_simpson_xgb$.estimate),
  `Prediction rate AUC` = c(overall_richness_xgb, overall_simpson_xgb)
) %>%
mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
knitr::kable(caption = "XGBoost classification performance:
                success rate vs. prediction rate AUC")
```

Table 4: XGBoost classification performance: success rate vs. prediction rate AUC

Outcome	Success rate AUC	Prediction rate AUC
Species richness	0.983	0.957
Simpson diversity	0.790	0.711

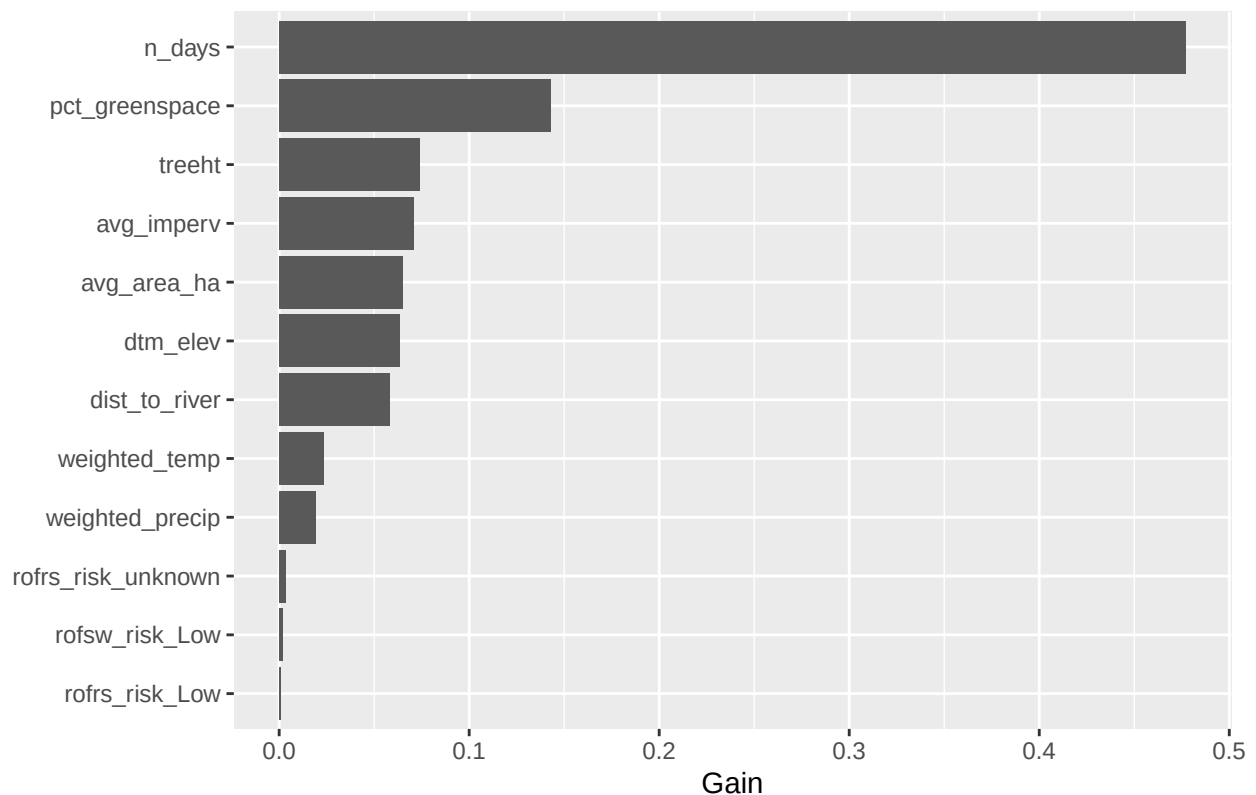
Variable Importance

```
richness_fit_xgb %>%  
  extract_fit_engine() %>%  
  xgboost::xgb.importance(model = .) %>%  
  as.data.frame() %>%  
  ggplot(aes(x = reorder(Feature, Gain), y = Gain)) +  
  geom_col() +  
  coord_flip() +  
  labs(title = "Variable importance: species richness classification (XGBoost)",  
        x = NULL, y = "Gain")
```



```
simpson_fit_xgb %>%  
  extract_fit_engine() %>%  
  xgboost::xgb.importance(model = .) %>%  
  as.data.frame() %>%  
  ggplot(aes(x = reorder(Feature, Gain), y = Gain)) +  
  geom_col() +  
  coord_flip() +  
  labs(title = "Variable importance: Simpson diversity classification (XGBoost)",  
        x = NULL, y = "Gain")
```

Variable importance: Simpson diversity classification (XGBoost)



XGBoost Without n_days

Set up and tuning

```
# Set new predictors without `n_days`
predictors_ND <- c("dtm_elev", "avg_imperv", "rofrsw_risk", "rofrs_risk",
  "avg_area_ha", "treeht", "weighted_precip", "weighted_temp",
  "dist_to_river", "pct_greenpace")

#Recipes and workflows, as before.
richness_recipe_xgb_ND <- recipe(
  as.formula(paste("richness_class ~", paste(predictors_ND, collapse = " + "))),
  data = df) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_naomit(all_numeric_predictors(), all_outcomes()) %>%
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors())

richness_wf_xgb_ND <- workflow() %>%
  add_recipe(richness_recipe_xgb_ND) %>%
  add_model(xgb_spec_class)

simpson_recipe_xgb_ND <- recipe(
```

```

as.formula(paste("simpson_class ~", paste(predictors_ND, collapse = " + "))),
data = df) %>%
step_unknown(all_nominal_predictors()) %>%
step_naomit(all_numeric_predictors(), all_outcomes()) %>%
step_novel(all_nominal_predictors()) %>%
step_dummy(all_nominal_predictors())

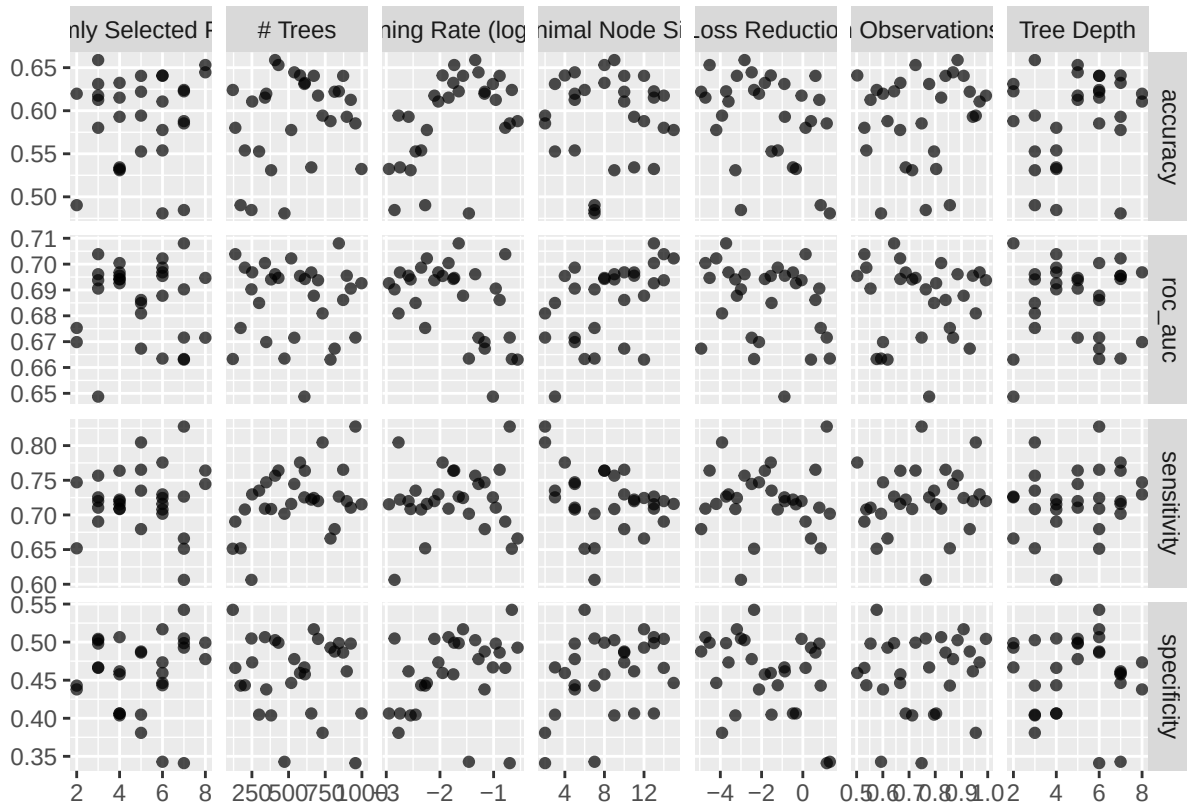
simpson_wf_xgb_ND <- workflow() %>%
  add_recipe(simpson_recipe_xgb_ND) %>%
  add_model(xgb_spec_class)

#Hyperparameter Tuning
richness_res_xgb_ND <- tune_grid(
  richness_wf_xgb_ND,
  resamples = folds,
  grid      = xgb_grid,
  metrics   = metric_set(roc_auc, accuracy, sensitivity, specificity))

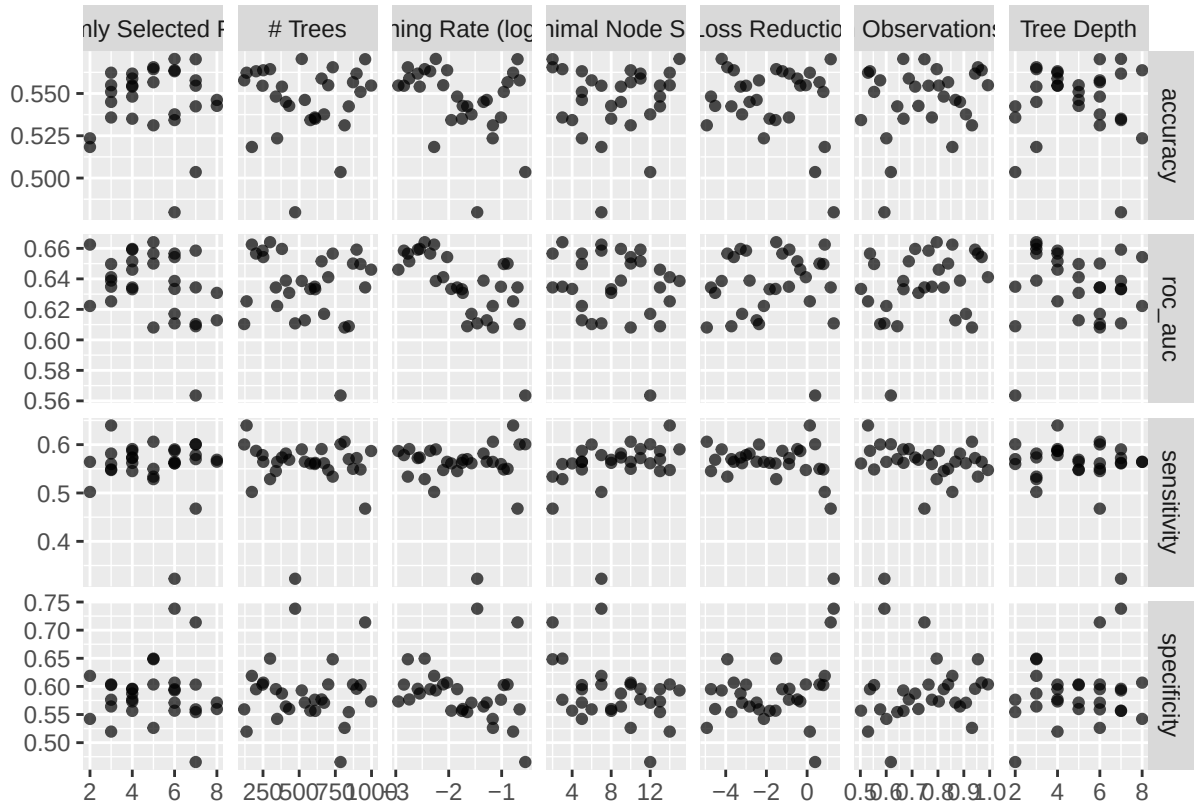
simpson_res_xgb_ND <- tune_grid(
  simpson_wf_xgb_ND,
  resamples = folds,
  grid      = xgb_grid,
  metrics   = metric_set(roc_auc, accuracy, sensitivity, specificity))

autoplot(richness_res_xgb_ND)

```



```
autoplot(simpson_res_xgb_ND)
```



```
#Select Best
show_best(richness_res_xgb_ND, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, tree_depth, learn_rate, loss_reduction,
         sample_size, trees, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(caption = "Top 5 hyperparameter combinations:
                 species richness ND class (XGBoost, ranked by ROC AUC)")
```

Table 5: Top 5 hyperparameter combinations: species richness ND class (XGBoost, ranked by ROC AUC)

mtry	min_n	tree_depth	learn_rate	loss_reduction	sample_size	trees	Mean ROC	
							AUC	Std. error
7	13	2	0.0223982	0.0001897	0.6429801	843	0.708	0.031
3	14	4	0.1647019	1.3640763	0.5297650	137	0.704	0.026
6	15	7	0.0057096	0.0000636	0.6659450	518	0.702	0.028
4	13	6	0.0144742	0.0000189	0.8224535	339	0.700	0.033
6	5	4	0.0044682	0.0609288	0.5372275	202	0.699	0.034

```
show_best(simpson_res_xgb_ND, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, tree_depth, learn_rate, loss_reduction,
         sample_size, trees, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(caption = "Top 5 hyperparameter combinations:
                    Simpson diversity ND class (XGBoost, ranked by ROC AUC)")
```

Table 6: Top 5 hyperparameter combinations: Simpson diversity
ND class (XGBoost, ranked by ROC AUC)

mtry	min_n	tree_depth	learn_rate	loss_reduction	sample_size	trees	Mean ROC AUC	Std. error
5	3	3	0.0035202	0.0304874	0.7953771	299	0.664	0.013
2	7	3	0.0053055	7.3573837	0.8556851	173	0.662	0.026
4	9	3	0.0028402	0.0005382	0.7119274	381	0.660	0.023
4	11	7	0.0026038	0.1345773	0.9434751	897	0.659	0.015
7	7	4	0.0014263	0.0009986	0.7637526	247	0.659	0.017

```
best_richness_xgb_ND <- select_best(richness_res_xgb_ND, metric = "roc_auc")
best_simpson_xgb_ND <- select_best(simpson_res_xgb_ND, metric = "roc_auc")
```

Prediction Rate AUC

```
overall_richness_xgb_ND <- collect_metrics(richness_res_xgb_ND) %>%
  filter(.metric == "roc_auc") %>%
  semi_join(best_richness_xgb_ND,
            by = names(best_richness_xgb_ND)) %>%
  pull(mean)

overall_simpson_xgb_ND <- collect_metrics(simpson_res_xgb_ND) %>%
  filter(.metric == "roc_auc") %>%
  semi_join(best_simpson_xgb_ND,
            by = names(best_simpson_xgb_ND)) %>%
  pull(mean)

city_richness_xgb_ND <- collect_metrics(
  richness_res_xgb_ND,
  summarize = FALSE) %>%
  filter(.metric == "roc_auc") %>%
  semi_join(best_richness_xgb_ND,
            by = names(best_richness_xgb_ND)) %>%
  left_join(fold_city_lookup, by = "id") %>%
  select(city, richness_auc = .estimate)

city_simpson_xgb_ND <- collect_metrics(
  simpson_res_xgb_ND,
  summarize = FALSE) %>%
  filter(.metric == "roc_auc") %>%
  left_join(fold_city_lookup, by = "id") %>%
  select(city, simpson_auc = .estimate)
```

```

semi_join(best_simpson_xgb_ND,
          by = names(best_simpson_xgb_ND)) %>%
left_join(fold_city_lookup, by = "id") %>%
select(city, simpson_auc = .estimate)

city_auc_xgb_ND <- city_richness_xgb_ND %>% full_join(city_simpson_xgb_ND, by = "city")

overall_auc_xgb_ND <- tibble(city = "Overall",
                             richness_auc = overall_richness_xgb_ND,
                             simpson_auc = overall_simpson_xgb_ND)

classification_auc_table_xgb_ND <- bind_rows(overall_auc_xgb_ND, city_auc_xgb_ND) %>%
mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
rename("City (Held-Out)" = city,
       "Richness ROC AUC" = richness_auc,
       "Simpson ROC AUC" = simpson_auc)

classification_auc_table_xgb_ND %>%
knitr::kable(caption = "Overall and city-level cross-validated ROC AUC for
ND species richness and Simpson diversity (XGBoost)")

```

Table 7: Overall and city-level cross-validated ROC AUC for ND species richness and Simpson diversity (XGBoost)

City (Held-Out)	Richness ROC AUC	Simpson ROC AUC
Overall	0.708	0.664
Nottingham	0.651	0.643
Exeter	0.676	0.667
Newcastle	0.666	0.631
Bristol	0.819	0.671
Gateshead	0.728	0.708

Final Model

```

final_richness_wf_xgb_ND <- finalize_workflow(richness_wf_xgb_ND, best_richness_xgb_ND)
richness_fit_xgb_ND <- fit(final_richness_wf_xgb_ND, data = df)

final_simpson_wf_xgb_ND <- finalize_workflow(simpson_wf_xgb_ND, best_simpson_xgb_ND)
simpson_fit_xgb_ND <- fit(final_simpson_wf_xgb_ND, data = df)

richness_recipe_xgb_ND_prepped <- extract_recipe(richness_fit_xgb_ND)
saveRDS(richness_recipe_xgb_ND_prepped, "Data/richness_recipe_xgb_ND.rds")
#saveRDS(richness_fit_xgb_ND, "richness_fit_xgb_ND.rds")

```

Success Rate AUC

```

richness_success_xgb_ND <- augment(richness_fit_xgb_ND, new_data = df)
simpson_success_xgb_ND <- augment(simpson_fit_xgb_ND, new_data = df)

```

```

success_rate_richness_xgb_ND <- roc_auc(richness_success_xgb_ND, truth = richness_class,
                                         .pred_high, event_level = "second")
success_rate_simpson_xgb_ND <- roc_auc(simpson_success_xgb_ND, truth = simpson_class,
                                         .pred_high, event_level = "second")

```

ROC Curves

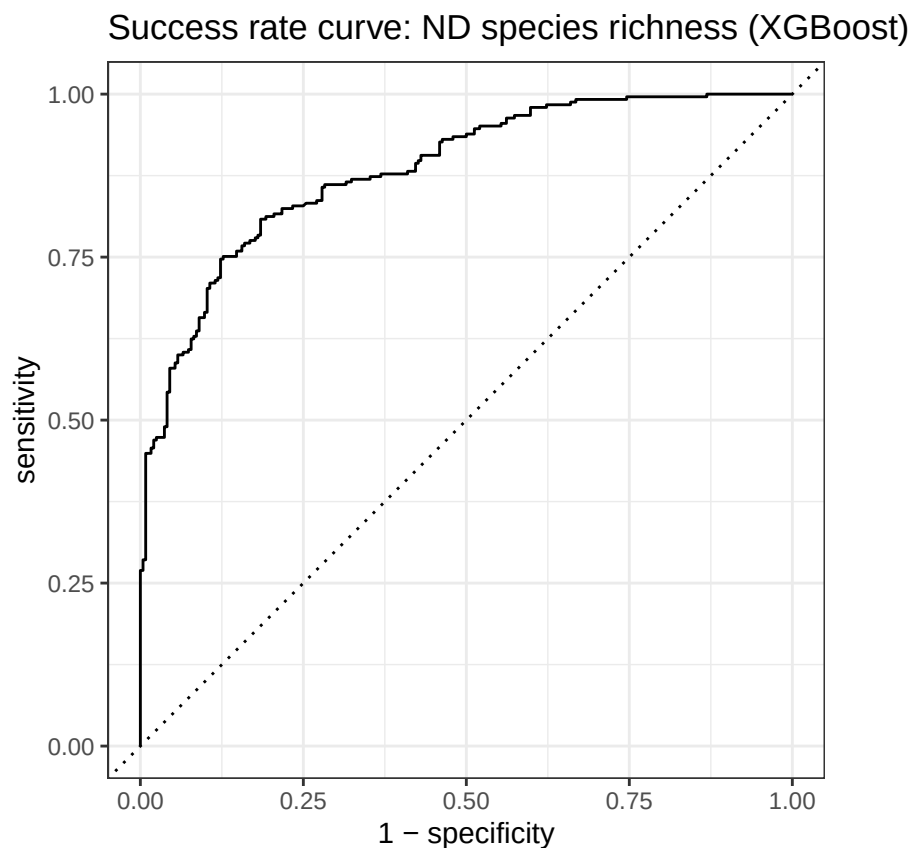
```

p1 <- roc_curve(richness_success_xgb_ND, truth = richness_class, .pred_high,
                event_level = "second") %>%
  autoplot() + labs(title = "Success rate curve: ND species richness (XGBoost)")

p2 <- roc_curve(simpson_success_xgb_ND, truth = simpson_class, .pred_high,
                event_level = "second") %>%
  autoplot() + labs(title = "Success rate curve: ND Simpson diversity (XGBoost)")

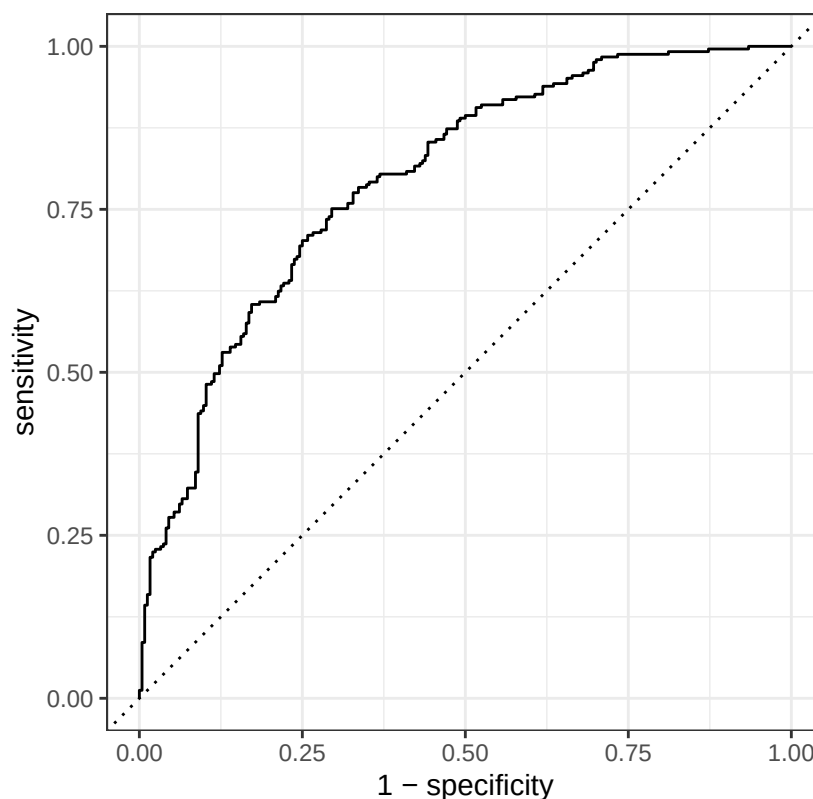
p1

```



p2

Success rate curve: ND Simpson diversity (XGBoost)



```
#ggsave("xgb_ND_rich_ROC.png", p1, width = 12, height = 5, dpi = 300)
#ggsave("xgb_ND_simp_ROC.png", p2, width = 12, height = 5, dpi = 300)
```

Summary Table

```
tibble::tibble(
  Outcome = c("ND Species richness", "ND Simpson diversity"),
  `Success rate AUC` = c(success_rate_richness_xgb_ND$.estimate,
                        success_rate_simpson_xgb_ND$.estimate),
  `Prediction rate AUC` = c(overall_richness_xgb_ND, overall_simpson_xgb_ND)
) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  knitr::kable(caption = "XGBoost classification performance:
                  ND success rate vs. ND prediction rate AUC")
```

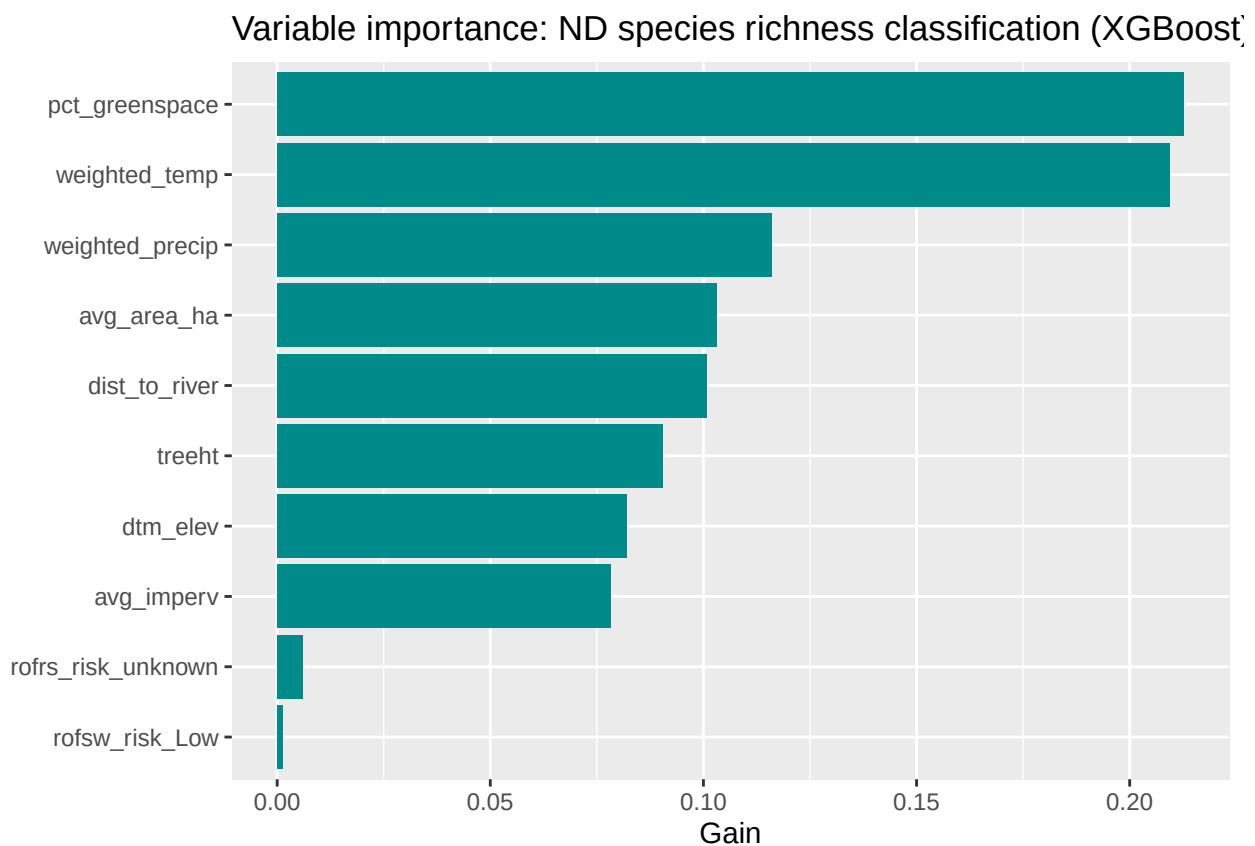
Table 8: XGBoost classification performance: ND success rate vs. ND prediction rate AUC

Outcome	Success rate AUC	Prediction rate AUC
ND Species richness	0.884	0.708
ND Simpson diversity	0.796	0.664

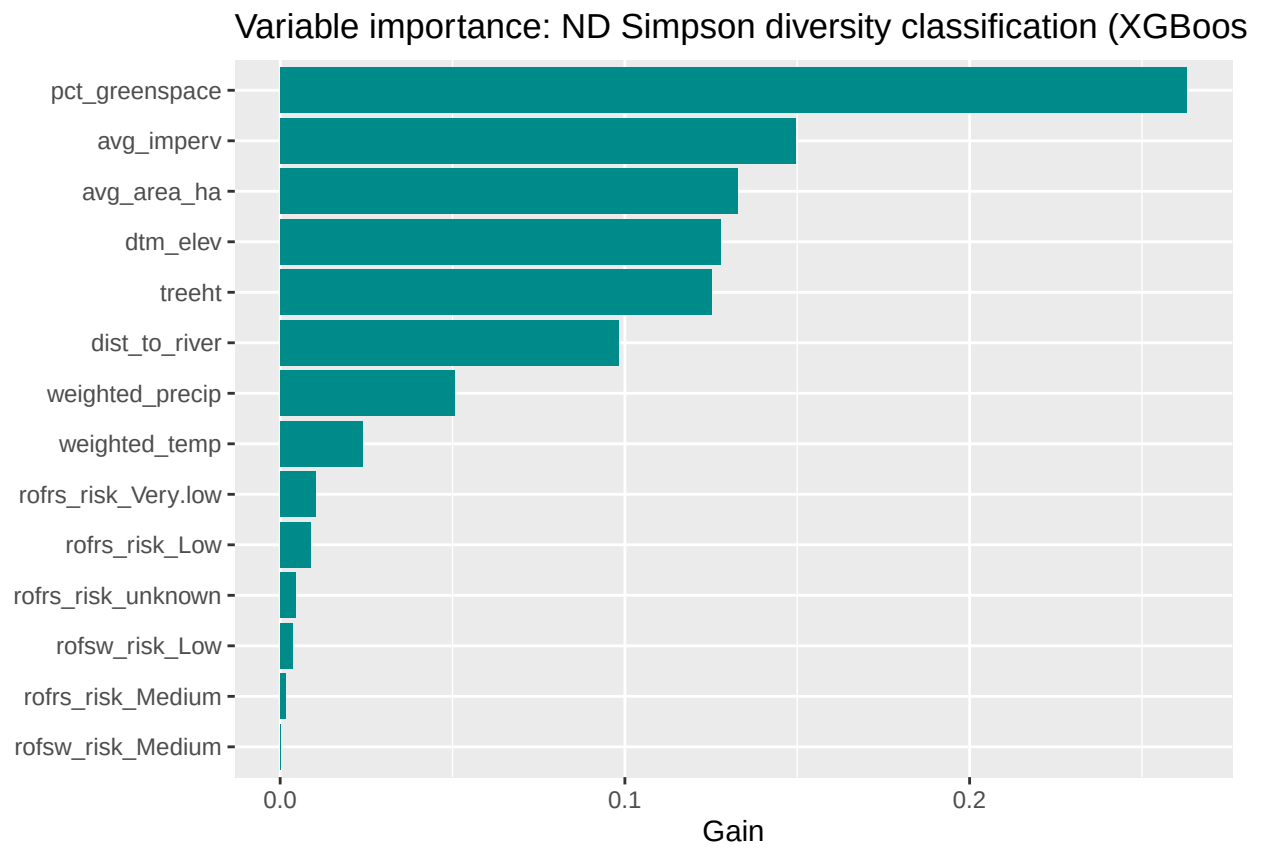
Variable Importance

```
p3 <- richness_fit_xgb_ND %>%  
  extract_fit_engine() %>%  
  xgboost::xgb.importance(model = .) %>%  
  as.data.frame() %>%  
  ggplot(aes(x = reorder(Feature, Gain), y = Gain)) +  
  geom_col(fill = "cyan4") +  
  coord_flip() +  
  labs(title = "Variable importance: ND species richness classification (XGBoost)",  
        x = NULL, y = "Gain")  
  
p4 <- simpson_fit_xgb_ND %>%  
  extract_fit_engine() %>%  
  xgboost::xgb.importance(model = .) %>%  
  as.data.frame() %>%  
  ggplot(aes(x = reorder(Feature, Gain), y = Gain)) +  
  geom_col(fill = "cyan4") +  
  coord_flip() +  
  labs(title = "Variable importance: ND Simpson diversity classification (XGBoost)",  
        x = NULL, y = "Gain")
```

p3



p4



```
#ggsave("VI_xgb_ND_rich.png", p3, width = 12, height = 5, dpi = 300)
#ggsave("VI_xgb_ND_simp.png", p4, width = 12, height = 5, dpi = 300)
```